



Security Assessment Report

Permissionless

Incremental Review

June 8, 2026

Summary

The Sec3 team was engaged to conduct a thorough security analysis of the Permissionless.

The artifact of the audit was the source code of the following programs, excluding tests, in a private repository.

The initial audit focused on the following versions and revealed 1 issues or questions.

Task	Type	Commit
Permissionless	Solana	1ffc0af07 (May 8, 2026) → a85eb0bf (May 26, 2026)

The post-audit review was conducted on commit ffa86e7 (June 5, 2026) in the release-v4.1.0-kamino-v2 branch.

This report provides a detailed description of the findings and their respective resolutions.

Table of Contents

Result Overview 3

Findings in Detail 4

 [I-01] Unconditional obligation initialization in Kamino strategy 4

Appendix: Methodology and Scope of Work 6

Result Overview

Issue	Impact	Status
PERMISSIONLESS		
[I-01] Unconditional obligation initialization in Kamino strategy	Info	Resolved

Findings in Detail

PERMISSIONLESS

[I-01] Unconditional obligation initialization in Kamino strategy

Identified in commit [a85eb0b](#).

The `setup_deployment_target` function within the `KaminoLendStrategy` is responsible for initializing the baseline accounts, including user metadata, obligation, and farm state. While the `init_user_metadata` CPI is gated by `if user_metadata_info.owner != KaminoLending::id()`, its execution of the `init_obligation` CPI is unconditional.

```
/* programs/permissionless/src/investment/strategies/kamino_lend_strategy.rs */
122 | fn setup_deployment_target<'info>(
...
158 |     if user_metadata_info.owner != &KaminoLending::id() {
...
171 |         cpi::init_user_metadata(cpi_ctx, Pubkey::default());
172 |     }
...
189 |     cpi::init_obligation(
190 |         cpi_ctx,
191 |         InitObligationArgs {
192 |             tag: OBLIGATION_TAG,
193 |             id: OBLIGATION_ID,
194 |         },
195 |     )?;
196 |
197 |     if let Some((
198 |         lending_market_authority_info,
199 |         reserve_farm_state_info,
200 |         obligation_farm_user_state_info,
201 |         farms_program_info,
202 |     )) = farm_accounts
203 |     {
...
221 |         cpi::init_obligation_farms_for_reserve(cpi_ctx, FARM_MODE_COLLATERAL)?;
222 |     }
223 |
224 |     Ok(true)
225 | }
```

While this implementation safely handles the standard operational flow, it introduces two specific corner cases that limit the protocol's future flexibility:

1. **Dynamic Farm Upgrade Friction:** Kamino allows lending reserves to dynamically activate farm re-wards after their initial creation. If a target reserve is integrated into the protocol before its farm is configured, this instruction will successfully create the Obligation without a `farm_user_state`. If

Kamino later activates a farm for that reserve, future Kamino V2 CPI calls will strictly demand the farm accounts. Because the `init_obligation` step lacks an idempotency guard (i.e., verifying if the Obligation is already initialized), administrators cannot simply re-invoke `setup_deployment_target` to append the new farm state.

2. **Isolated Multi-Reserve Deployments:** Kamino's native architecture maps one obligation to one lending market, but allows that single obligation to hold positions across multiple deposit reserves. If the pool protocol expands to deploy liquidity into a second reserve within the same lending market, it would need to invoke `setup_deployment_target` again to trigger `init_obligation_farms_for_reserve` for the new reserve. However, because the Obligation was already created during the first reserve's setup, the unconditional initialization step will revert, preventing the second reserve's farm from being successfully registered.

Given that Huma pools typically operate with a fixed underlying mint, these limitations are unlikely to impact current operations. As a defense-in-depth measure, consider refactoring the `init_obligation` logic to be idempotent.

Resolution

This issue has been resolved by `df7ce18`.

Appendix: Methodology and Scope of Work

Assisted by the Sec3 Scanner developed in-house, the manual audit particularly focused on the following work items:

- Check common security issues.
- Check program logic implementation against available design specifications.
- Check poor coding practices and unsafe behavior.
- The soundness of the economics design and algorithm is out of scope of this work

DISCLAIMER

The instance report ("Report") was prepared pursuant to an agreement between Coderrect Inc. d/b/a Sec3 (the "Company") and Huma Technologies LLC (the "Client"). This Report solely includes the results of a technical assessment of a specific build and/or version of the Client's code specified in the Report ("Assessed Code") by the Company. The sole purpose of the Report is to provide the Client with the results of the technical assessment of the Assessed Code. The Report does not apply to any other version and/or build of the Assessed Code. Regardless of the contents of the Report, the Report does not (and should not be interpreted to) provide any warranty, representation or covenant that the Assessed Code: (i) is error and/or bug free, (ii) has no security vulnerabilities, and/or (iii) does not infringe any third-party rights. Moreover, the Report is not, and should not be considered, an endorsement by the Company of the Assessed Code and/or of the Client. Finally, the Report should not be considered investment advice or a recommendation to invest in the Assessed Code and/or the Client.

This Report is considered null and void if the Report (or any portion thereof) is altered in any manner.

ABOUT

The Sec3 audit team comprises a group of computer science professors, researchers, and industry veterans with extensive experience in smart contract security, program analysis, testing, and formal verification. We are also building automated security tools that incorporate static analysis, penetration testing, and formal verification.

At Sec3, we identify and eliminate security vulnerabilities through the most rigorous process and aided by the most advanced analysis tools.

For more information, check out our [website](#) and follow us on [twitter](#).

